LỜI GIẢI THAM KHẢO
CHUYÊN ĐỀ TÌM KIẾM NHỊ PHÂN

Huỳnh Tấn Đức - ĐH Khoa Học Tự Nhiên, ĐHQGHCM

Bài 1: Truy vấn giới hạn trên (Last Element $\le x$)

1. Phân tích bài toán

- **Đặc điểm dữ liệu:** Đề bài cho mảng A đã được sắp xếp theo thứ tự không giảm. Đây là dấu hiệu quan trọng nhất báo hiệu ta có thể áp dụng thuật toán Tìm kiếm nhị phân (Binary Search) để tối ưu hóa thời gian tìm kiếm từ $O(n)$ xuống còn $O(\log n)$.
- **Mục tiêu:** Với mỗi số x, tìm vị trí i **lớn nhất** sao cho $A_i \le x$. Nếu không có phần tử nào thỏa mãn (tức là mọi phần tử đều $> x$), trả về -1.

2. Xây dựng từng bước

Để tìm vị trí phần tử cuối cùng $\le x$, ta thiết lập hai con trỏ `left` và `right` ở hai đầu mảng. Trong mỗi bước lặp:

- Tính vị trí giữa: `mid = (left + right) / 2`.
- **Trường hợp $A[\textbf{mid}] \le x$:** Phần tử tại `mid` thỏa mãn điều kiện. Tuy nhiên, vì ta cần tìm vị trí **lớn nhất**, ta lưu `ans = mid` và tiếp tục dịch con trỏ `left = mid + 1` để tìm kiếm thêm ở nửa bên phải xem có vị trí nào tốt hơn không.
- **Trường hợp $A[\textbf{mid}] > x$:** Phần tử này quá lớn, ta bắt buộc phải tìm ở nửa bên trái bằng cách ép `right = mid - 1`.

```cpp
int solve(int x) {
    int left = 1, right = n;
    int ans = -1;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (a[mid] <= x) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}
```

3. Mã nguồn hoàn chỉnh Thuật toán duyệt qua q truy vấn, mỗi truy vấn mất $O(\log n)$ thời gian. Tổng độ phức tạp là $O(q \log n)$, cực kỳ tối ưu và dễ dàng vượt qua giới hạn 1.0s.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, q;
int a[100005];

int solve(int x) {
    int left = 1, right = n;
    int ans = -1;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (a[mid] <= x) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return ans;
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> q) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }

        while (q--) {
            int x;
            cin >> x;
            cout << solve(x) << '\n';
        }
    }

    return 0;
}
```

Bài 2: Đếm số lượng dãy con liên tiếp

1. Phân tích bài toán

- **Đặc tính mảng:** Vì tất cả phần tử trong mảng A đều là số nguyên dương ($A_i \ge 1$), nên nếu ta xây dựng Mảng cộng dồn (`Prefix Sum`) P, mảng P này sẽ là một dãy số tăng ngặt. Tính chất này cho phép ta áp dụng Tìm kiếm nhị phân trực tiếp trên mảng cộng dồn.
- **Biến đổi toán học:** Tổng của một dãy con liên tiếp từ vị trí j đến vị trí i (với $j \le i$) được tính bằng công thức: $P_i - P_{j-1}$.
- Yêu cầu bài toán là đếm số lượng dãy con có tổng $\ge k$:

$$P_i - P_{j-1} \ge k \iff P_{j-1} \le P_i - k$$

- Như vậy, với mỗi vị trí i (từ 1 đến N), ta chỉ cần đếm xem có bao nhiêu chỉ số $j-1$ (từ 0 đến $i-1$) thỏa mãn giá trị tại đó $\le P_i - k$.

2. Xây dựng từng bước

Bước 1: Khởi tạo mảng cộng dồn Ta tính mảng P với $P_0 = 0$. $P_i = P_{i-1} + A_i$. Lưu ý: Tổng mảng có thể lên tới $10^5 \times 10^9 = 10^{14}$, nên mảng P bắt buộc phải sử dụng kiểu `long long`.

Bước 2: Tìm kiếm nhị phân bằng hàm có sẵn Để đếm số lượng phần tử $\le P_i - k$ trong đoạn từ 0 đến $i-1$, ta có thể sử dụng hàm `upper_bound` tích hợp sẵn của C++.

- `upper_bound(P, P + i, P[i] - k)` sẽ trả về con trỏ tới phần tử đầu tiên **lớn hơn** $P_i - k$.
- Khoảng cách từ vị trí bắt đầu mảng P đến con trỏ này chính là số lượng các phần tử thỏa mãn điều kiện $\le P_i - k$. Ta chỉ cần cộng dồn khoảng cách này vào kết quả.

```cpp
int ans = 0;
for (int i = 1; i <= n; i++) {
    int target = pre[i] - k;
    int pos = upper_bound(pre, pre + i, target) - pre;
    ans += pos;
}
```

3. Mã nguồn hoàn chỉnh Số lượng dãy con thỏa mãn trong trường hợp tốt nhất có thể lên tới $\frac{N(N+1)}{2} \approx 5 \cdot 10^9$, do đó biến kết quả `ans` cũng phải được khai báo bằng `long long`. Độ phức tạp thuật toán là $O(N \log N)$, cực kỳ mượt mà.

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, k;
int a[100005];
int pre[100005];

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> k) {
        pre[0] = 0;
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
            pre[i] = pre[i - 1] + a[i];
        }

        int ans = 0;
        for (int i = 1; i <= n; i++) {
            int target = pre[i] - k;
            int pos = upper_bound(pre, pre + i, target) - pre;
            ans += pos;
        }

        cout << ans << '\n';
    }

    return 0;
}
```

Bài 3: Thống kê tần số trong khoảng (Range Frequency Query)

1. Phân tích bài toán & Ý tưởng

- **Cách ngây thơ (Brute-force):** Với mỗi truy vấn, ta duyệt mảng từ vị trí l đến r để đếm số lần xuất hiện của x. Thời gian cho mỗi truy vấn là $O(n)$, tổng thời gian là $O(n \times q)$. Với $n, q \le 10^5$, phương pháp này tốn khoảng 10^{10} phép toán và chắc chắn sẽ bị quá thời gian (Time Limit Exceeded - TLE).
- **Cách tối ưu:** Ta nhận thấy cấu trúc mảng là tĩnh (không có thao tác cập nhật phần tử) và mục tiêu là đếm số lượng vị trí của một giá trị x cụ thể nằm trong đoạn $[l, r]$. Thay vì duyệt lại mảng nhiều lần, ta có thể phân nhóm và lưu trước các vị trí xuất hiện của mỗi giá trị.

2. Thuật toán

- **Tiền xử lý:** Sử dụng một mảng các `vector` (ở đây là `id[]`) để lưu lại tập hợp các vị trí của từng giá trị. Khi đọc mảng A từ 1 đến n, nếu phần tử tại vị trí i có giá trị x (`ai`), ta thêm vị trí i vào cuối `id[x]`. Do ta duyệt mảng theo chiều từ trái sang phải, các vị trí trong mỗi `vector id[x]` sẽ tự động được lưu theo thứ tự sắp xếp tăng dần.
- **Trả lời truy vấn:** Với mỗi bộ (l, r, x), ta cần đếm xem trong mảng `id[x]` có bao nhiêu phần tử mang giá trị $\ge l$ và $\le r$. Vì `id[x]` đã được sắp xếp, ta có thể ứng dụng tìm kiếm nhị phân (Binary Search) tích hợp sẵn trong C++:
  - Sử dụng `lower_bound(..., l)` để tìm *iterator* trỏ tới phần tử đầu tiên $\ge l$.
  - Sử dụng `upper_bound(..., r)` để tìm *iterator* trỏ tới phần tử đầu tiên $> r$.
  - Phép trừ giữa hai *iterator* này sẽ trả về chính xác số lượng phần tử thỏa mãn điều kiện nằm trong đoạn $[l, r]$.

3. Độ phức tạp

- **Độ phức tạp thời gian (Time Complexity):** Bước tiền xử lý đọc dữ liệu mất $O(n)$. Mỗi truy vấn thực hiện hai lần tìm kiếm nhị phân trên vector có kích thước tối đa là n, mất thời gian $O(\log n)$. Vậy tổng thời gian cho toàn bộ chương trình là $O(n + q \log n)$, cực kỳ tối ưu và dễ dàng vượt qua giới hạn 1000 ms.
- **Độ phức tạp không gian (Space Complexity):** Mảng `vector id` có kích thước dựa trên giới hạn của phần tử (10^5), và tổng số lượng phần tử được đẩy vào các vector đúng bằng n. Độ phức tạp không gian là $O(n + \max(A_i))$, hoàn toàn an toàn trong giới hạn bộ nhớ 256 MB.

4. Mã nguồn tham khảo

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> id[100005];

signed main(){
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++){
        int ai; cin >> ai;
        id[ai].push_back(i);
    }

    while (q--){
        int l, r, x;
        cin >> l >> r >> x;
        // So luong phan tu trong [l, r] = So phan tu <= r tru di so
  phan tu < l
        cout << upper_bound(id[x].begin(), id[x].end(), r)
               - lower_bound(id[x].begin(), id[x].end(), l) << '\n';
    }

    return 0;
}
```

Bài 4: Phân loại kẹo trên băng chuyền

1. Phân tích bài toán

- **Khó khăn:** Tổng số lượng kẹo trên băng chuyền có thể lên tới 10^{14} (tổng các a_i). Nếu tạo một mảng lưu từng viên kẹo theo cách mô phỏng thông thường, chương trình sẽ ngay lập tức bị tràn bộ nhớ (Memory Limit Exceeded) và quá thời gian (Time Limit Exceeded).
- **Hướng tiếp cận:** Thay vì lưu từng viên kẹo, ta sẽ quản lý kẹo theo từng *nhóm* (loại kẹo). Băng chuyền yêu cầu sắp xếp kẹo theo khối lượng không giảm, do đó ta cần sắp xếp các nhóm kẹo này theo khối lượng trước, sau đó dùng kỹ thuật **Mảng cộng dồn (Prefix Sum)** kết hợp **Tìm kiếm nhị phân (Binary Search)** để xác định vị trí.

2. Ý tưởng thuật toán

- **Bước 1 - Biểu diễn dữ liệu:** Lưu thông tin của mỗi loại kẹo dưới dạng một cặp giá trị `pair<int, int>` gồm số lượng (a_i) và khối lượng (w_i).
- **Bước 2 - Sắp xếp:** Sắp xếp mảng các loại kẹo theo chiều tăng dần của khối lượng.
- **Bước 3 - Mảng cộng dồn:** Xây dựng mảng `cnt` với `cnt[i]` là tổng số lượng kẹo của i loại nhẹ nhất đầu tiên.
  - Lúc này, các viên kẹo thuộc loại thứ i (sau khi đã sắp xếp) sẽ nằm ở các vị trí từ $\texttt{cnt}[i-1] + 1$ cho đến $\texttt{cnt}[i]$ trên băng chuyền.
  - Vì $a_i \ge 1$, mảng `cnt` là một mảng tăng ngặt.
- **Bước 4 - Truy vấn:** Với mỗi vị trí k cần tìm, ta chỉ cần tìm chỉ số i nhỏ nhất thỏa mãn $\texttt{cnt}[i] \ge k$. Do `cnt` đã được sắp xếp tăng dần, ta có thể dùng hàm `lower_bound` để tìm kiếm nhanh trong thời gian $O(\log n)$. Khối lượng của viên kẹo tại vị trí đó chính là khối lượng của loại kẹo thứ i.

3. Độ phức tạp

- **Thời gian:** Sắp xếp n loại kẹo tốn $O(n \log n)$. Tính mảng cộng dồn tốn $O(n)$. Xử lý q truy vấn, mỗi truy vấn tìm kiếm nhị phân tốn $O(\log n)$, tổng cộng là $O(q \log n)$. Thời gian tổng quát là $O((n+q)\log n)$, chạy cực nhanh và thoải mái qua giới hạn 1.0s.
- **Không gian:** Lưu trữ mảng `pair` và mảng cộng dồn `cnt` kích thước n tiêu tốn $O(n)$ bộ nhớ, hoàn toàn thỏa mãn giới hạn 256MB.

4. Mã nguồn tham khảo

```cpp
#include <bits/stdc++.h>
using namespace std;
#define ii pair<int,int>
#define fi first
#define se second

const int N = 1e5 + 31 + 01 + 2007;
ii a[N];
long long cnt[N];

bool cmp(ii x, ii y){
    return x.se < y.se;
}

signed main(){
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    int n, q;
    cin >> n >> q;
    for (int i = 1; i <= n; i++)
        cin >> a[i].fi >> a[i].se;

    // Sap xep theo khoi luong
    sort(a + 1, a + n + 1, cmp);

    // Tinh mang cong don so luong keo
    for (int i = 1; i <= n; i++)
        cnt[i] = cnt[i - 1] + a[i].fi;

    // Query
    while (q--){
        long long k;
        cin >> k;
        // Tim vi tri dau tien co cnt >= k
        int pos = lower_bound(cnt + 1, cnt + n + 1, k) - cnt;
        cout << a[pos].se << '\n';
    }

    return 0;
}
```

Bài 5: Định vị số Hamming

1. Phân tích bài toán

- Một số Hamming có dạng

$$H = 2^x \cdot 3^y \cdot 5^z$$

- Giới hạn của bài toán là $m \le 10^{18}$. Ta thử ước lượng số mũ tối đa của từng cơ số:
  - $2^x \le 10^{18} \Rightarrow x \le 60$
  - $3^y \le 10^{18} \Rightarrow y \le 38$
  - $5^z \le 10^{18} \Rightarrow z \le 26$
- Số lượng các tổ hợp (x, y, z) thỏa mãn điều kiện $H \le 10^{18}$ là rất nhỏ (khoảng hơn 60.000 số). Thay vì đếm chay cho từng truy vấn, ta hoàn toàn có thể sinh ra toàn bộ các số Hamming trong giới hạn này, lưu vào một mảng, sắp xếp lại và tìm kiếm.

2. Xây dựng từng bước

Bước 1: Sinh toàn bộ số Hamming Ta dùng ba vòng lặp lồng nhau đại diện cho các lũy thừa của 2, 3 và 5. Cần lưu ý điều kiện dừng để không bị tràn kiểu dữ liệu (Overflow) khi nhân lên. Sau khi sinh xong, ta sắp xếp lại mảng `H` theo thứ tự tăng dần.

```cpp
vector<int> H;

void sinh()
{
    int MAX = 1e18;
    for (int p2 = 1; p2 <= MAX; ) {
        for (int p3 = 1; p3 <= MAX / p2; ) {
            for (int p5 = 1; p5 <= MAX / (p2 * p3); ) {
                H.push_back(p2 * p3 * p5);
                if (p5 > MAX / 5) break;
                p5 *= 5;
            }
            if (p3 > MAX / 3) break;
            p3 *= 3;
        }
        if (p2 > MAX / 2) break;
        p2 *= 2;
    }
    sort(H.begin(), H.end());
}
```

Bước 2: Xử lý truy vấn bằng Tìm kiếm nhị phân Mảng `H` hiện tại đã chứa toàn bộ số Hamming tăng dần. Với mỗi truy vấn m, ta dùng hàm `lower_bound` để tìm vị trí xuất hiện của m. Nếu giá trị tại vị trí tìm được đúng bằng m, ta in ra chỉ số đó (cộng 1 vì `vector` bắt đầu từ 0). Nếu không, in ra -1.

```cpp
int m;
cin >> m;
int pos = lower_bound(H.begin(), H.end(), m) - H.begin();

if (pos < H.size() && H[pos] == m) cout << pos + 1 << '\n';
else cout << -1 << '\n';
```

3. Mã nguồn hoàn chỉnh

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

vector<int> H;

void sinh()
{
    int MAX = 1e18;
    for (int p2 = 1; p2 <= MAX; ) {
        for (int p3 = 1; p3 <= MAX / p2; ) {
            for (int p5 = 1; p5 <= MAX / (p2 * p3); ) {
                H.push_back(p2 * p3 * p5);
                if (p5 > MAX / 5) break;
                p5 *= 5;
            }
            if (p3 > MAX / 3) break;
            p3 *= 3;
        }
        if (p2 > MAX / 2) break;
        p2 *= 2;
    }
    sort(H.begin(), H.end());
}

signed main()
{
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    sinh();

    int q;
    cin >> q;
    while (q--)
    {
        int m;
        cin >> m;
        int pos = lower_bound(H.begin(), H.end(), m) - H.begin();

        if (pos < H.size() && H[pos] == m) cout << pos + 1 << '\n';
        else cout << -1 << '\n';
    }

    return 0;
}
```

Bài 6: Định vị chuỗi số chia hết (Divisibility Sequence)

1. Phân tích bài toán

- **Khó khăn:** Giới hạn $k \leq 10^{14}$ vô cùng lớn, ta không thể tạo mảng hay đếm từng số một.
- **Đổi góc nhìn:** Thay vì tìm "số thứ k là số nào", ta giải quyết bài toán ngược: "Cho trước số x, có bao nhiêu số nhỏ hơn hoặc bằng x chia hết cho 2, 3 hoặc 5?". Gọi hàm này là $F(x)$.
- **Tính đơn điệu:** Rõ ràng khi x tăng lên thì $F(x)$ cũng tăng lên (hoặc không đổi). Tính chất đơn điệu này cho phép ta áp dụng **Tìm kiếm nhị phân (Binary Search)** để tìm ra số x nhỏ nhất thỏa mãn $F(x) = k$.

2. Xây dựng từng bước

Bước 1: Hàm đếm bằng Nguyên lý bù trừ (Inclusion-Exclusion) Để tính số lượng các số $\leq x$ chia hết cho ít nhất một trong ba số 2, 3, 5, ta không thể chỉ cộng đơn thuần $\lfloor \frac{x}{2} \rfloor + \lfloor \frac{x}{3} \rfloor + \lfloor \frac{x}{5} \rfloor$ vì các số chia hết cho nhiều số (ví dụ 6 chia hết cho cả 2 và 3) sẽ bị đếm trùng nhiều lần.

Áp dụng Nguyên lý bù trừ, ta có công thức chính xác:

$$F(x) = \lfloor \frac{x}{2} \rfloor + \lfloor \frac{x}{3} \rfloor + \lfloor \frac{x}{5} \rfloor - \lfloor \frac{x}{6} \rfloor - \lfloor \frac{x}{10} \rfloor - \lfloor \frac{x}{15} \rfloor + \lfloor \frac{x}{30} \rfloor$$

Trong đó 6, 10, 15 là BCNN của từng cặp, và 30 là BCNN của cả ba số (2, 3, 5).

```cpp
int countDiv(int x)
{
    return (x / 2) + (x / 3) + (x / 5)
         - (x / 6) - (x / 10) - (x / 15)
         + (x / 30);
}
```

Bước 2: Tìm kiếm nhị phân tìm giá trị x Ta thiết lập khoảng tìm kiếm $[L, R]$. Mức thấp nhất $L = 1$. Để xác định R, ta nhẩm tính: trong mỗi khoảng 30 số liên tiếp, có chính xác 22 số chia hết cho ít nhất một trong ba số (2, 3, 5). Do đó, giá trị của số thứ k sẽ rơi vào khoảng xấp xỉ $\frac{30}{22}k \approx 1.36k$. Với $k \leq 10^{14}$, ta có thể đặt $R = 2 \cdot 10^{14}$ là hoàn toàn an toàn và bao trọn nghiệm.

```cpp
int solve(int k)
{
    int left = 1, right = 2e14;
    int ans = 0;

    while (left <= right)
    {
        int mid = (left + right) / 2;
        if (countDivisible(mid) >= k) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return ans;
}
```

3. Mã nguồn hoàn chỉnh Kết hợp các bước trên lại để xử lý luồng truy vấn. Nhớ sử dụng `long long` (được `define` thành `int`) vì x và k có thể lên tới 10^{14}, vượt qua giới hạn của số nguyên 32-bit thông thường.

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int countDivisible(int x)
{
    return (x / 2) + (x / 3) + (x / 5)
         - (x / 6) - (x / 10) - (x / 15)
         + (x / 30);
}

int solve(int k)
{
    int left = 1, right = 2e14;
    int ans = 0;

    while (left <= right)
    {
        int mid = (left + right) / 2;
        if (countDiv(mid) >= k) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    return ans;
}

signed main()
{
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    int q;
    cin >> q;
    while (q--)
    {
        int k;
        cin >> k;
        cout << solve(k) << '\n';
    }

    return 0;
}
```

Bài 7: Thiết lập lõi năng lượng (Max of Min Distance)

1. Phân tích bài toán

- **Dấu hiệu nhận biết:** Cụm từ "khoảng cách ngắn nhất ... là lớn nhất có thể" (Maximize the Minimum) là dấu hiệu đặc trưng nhất của kỹ thuật Tìm kiếm nhị phân trên tập kết quả.
- **Hướng tiếp cận:** Thay vì cố gắng tìm trực tiếp cách đặt k lõi phản ứng, ta sẽ lật ngược vấn đề: "Giả sử ta muốn khoảng cách tối thiểu giữa 2 lõi bất kỳ là d, liệu ta có thể đặt được ít nhất k lõi phản ứng vào các khoang trống hay không?".
- Rõ ràng, nếu khoảng cách d càng lớn thì số lõi đặt được càng ít. Tính đơn điệu này cho phép ta dùng Tìm kiếm nhị phân để tìm giá trị d lớn nhất thỏa mãn.

2. Xây dựng từng bước

Bước 1: Sắp xếp và Hàm kiểm tra (Greedy Check) Trước tiên, ta phải sắp xếp tọa độ các khoang chứa theo thứ tự tăng dần. Hàm kiểm tra `check(mid)` sẽ cố gắng đặt các lõi phản ứng từ trái sang phải bằng thuật toán Tham lam. Ta đặt lõi đầu tiên ở khoang đầu tiên, sau đó duyệt qua các khoang tiếp theo. Nếu khoang nào có khoảng cách so với lõi gần nhất vừa đặt $\geq mid$, ta sẽ đặt thêm một lõi ở đó. Nếu tổng số lõi đặt được $\geq k$, mức khoảng cách `mid` này là khả thi.

```cpp
int n, k;
int a[100005];

bool check(int mid)
{
    int cnt = 1;
    int last_pos = a[1];

    for (int i = 2; i <= n; i++)
    {
        if (a[i] - last_pos >= mid)
        {
            cnt++;
            last_pos = a[i];
        }
    }
    return cnt >= k;
}
```

Bước 2: Tìm kiếm nhị phân giá trị khoảng cách Khoảng cách nhỏ nhất có thể là 1. Khoảng cách lớn nhất có thể là khoảng cách giữa khoang đầu và khoang cuối cùng ($A_n - A_1$). Ta sẽ tìm kiếm nhị phân trong đoạn này để tìm ra giá trị khả thi lớn nhất.

```cpp
int solve()
{
    int left = 1, right = a[n] - a[1];
    int ans = 0;

    while (left <= right)
    {
        int mid = (right + left) / 2;
        if (check(mid))
        {
            ans = mid;
            left = mid + 1;
        }
        else
        {
            right = mid - 1;
        }
    }
    return ans;
}
```

3. Mã nguồn hoàn chỉnh Kết hợp logic lại, ta có chương trình hoàn chỉnh. Thuật toán có độ phức tạp là $O(N \log N + N \log(\max A_i))$, chạy dư sức qua giới hạn thời gian 1.0s.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, k;
int a[100005];

bool check(int mid)
{
    int cnt = 1;
    int last_pos = a[1];

    for (int i = 2; i <= n; i++)
    {
        if (a[i] - last_pos >= mid)
        {
            cnt++;
            last_pos = a[i];
        }
    }
    return cnt >= k;
}

int solve()
{
    int left = 1, right = a[n] - a[1];
    int ans = 0;

    while (left <= right)
    {
        int mid = (right + left) / 2;
        if (check(mid))
        {
            ans = mid;
            left = mid + 1;
        }
        else
        {
            right = mid - 1;
        }
    }
    return ans;
}

signed main()
{
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    cin >> n >> k;
    for (int i = 1; i <= n; i++)
    {
        cin >> a[i];
    }

    sort(a + 1, a + n + 1);

    cout << solve() << '\n';

    return 0;
}
```

Bài 8: Lắp đặt trạm phát sóng Wifi (Minimize the Maximum)

1. Phân tích bài toán

- **Dấu hiệu:** Yêu cầu tìm "giá trị R nhỏ nhất sao cho tất cả được phủ sóng" chính là dấu hiệu của dạng toán Tìm kiếm nhị phân trên tập kết quả.
- **Tính đơn điệu:** Rõ ràng nếu bán kính R càng lớn thì một trạm sẽ phủ sóng được càng nhiều hộ dân, dẫn đến tổng số trạm cần dùng càng ít.
- Nếu với một bán kính R, ta có thể phủ sóng toàn bộ với số trạm $\leq K$, ta sẽ cố gắng thu hẹp R lại để tìm mức tối ưu hơn. Ngược lại, nếu cần $> K$ trạm, ta bắt buộc phải tăng R lên.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Greedy Check) Ta cần kiểm tra xem với bán kính R cho trước, ta cần ít nhất bao nhiêu trạm. Một trạm có bán kính R sẽ phủ sóng một đoạn có tổng chiều dài là $2R$. Thuật toán Tham lam được áp dụng như sau: Đứng tại hộ dân đầu tiên chưa có sóng (ở tọa độ x), để tối ưu, ta đặt trạm ở vị trí $x + R$. Khi đó, trạm sẽ phủ sóng xa nhất đến tọa độ $x + 2R$. Tất cả các hộ dân nằm trong khoảng $[x, x + 2R]$ đều được an toàn bỏ qua.

```cpp
int n, k;
int x[100005];

bool check(int mid)
{
    int cnt = 0;
    int i = 1;
    while (i <= n)
    {
        cnt++;
        int limit = x[i] + 2 * mid;
        while (i <= n && x[i] <= limit)
        {
            i++;
        }
    }
    return cnt <= k;
}
```

Bước 2: Tìm kiếm nhị phân bán kính Mức bán kính nhỏ nhất có thể là 0 (nếu số trạm $K \geq N$, mỗi nhà 1 trạm). Mức bán kính lớn nhất có thể là khoảng cách giữa nhà cuối và nhà đầu ($x_n - x_1$). Ta sẽ chặt nhị phân trong khoảng này. Khác với Bài 7, nếu `check(mid)` đúng, ta lưu kết quả và ép `right = mid - 1` để tìm R nhỏ hơn.

```cpp
int solve()
{
    int left = 0, right = x[n] - x[1];
    int ans = right;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (check(mid))
        {
            ans = mid;
            right = mid - 1;
        }
        else
        {
            left = mid + 1;
        }
    }
    return ans;
}
```

3. Mã nguồn hoàn chỉnh Độ phức tạp của bài toán là $O(N \log(\max X))$, tốc độ này hoàn toàn nằm trong vùng an toàn tuyệt đối với giới hạn của bài. Do mảng tọa độ đã được đề bài sắp xếp tăng dần sẵn, ta không cần phải gọi hàm `sort` nữa.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, k;
int x[100005];

bool check(int mid)
{
    int cnt = 0;
    int i = 1;
    while (i <= n)
    {
        cnt++;
        int limit = x[i] + 2 * mid;
        while (i <= n && x[i] <= limit)
        {
            i++;
        }
    }
    return cnt <= k;
}

int solve()
{
    int left = 0, right = x[n] - x[1];
    int ans = right;

    while (left <= right)
    {
        int mid = left + (right - left) / 2;
        if (check(mid))
        {
            ans = mid;
            right = mid - 1;
        }
        else
        {
            left = mid + 1;
        }
    }
    return ans;
}

signed main()
{
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    cin >> n >> k;
    for (int i = 1; i <= n; i++)
    {
        cin >> x[i];
    }

    cout << solve() << '\n';

    return 0;
}
```

Bài 9: Cân bằng hệ thống (Min of Max Sum)

1. Phân tích bài toán

- **Dấu hiệu:** Cụm từ "mức dung lượng lớn nhất ... đạt giá trị nhỏ nhất" (Minimize the Maximum) tiếp tục là lời gọi mời cho kỹ thuật Tìm kiếm nhị phân trên kết quả.
- **Tính đơn điệu:** Giả sử ta ấn định một mức dung lượng tối đa cho mỗi phân đoạn là x. Nếu x càng lớn, số đoạn ta cần chia sẽ càng ít. Ngược lại, nếu x càng nhỏ, ta sẽ phải cắt nhỏ luồng dữ liệu ra thành rất nhiều đoạn.
- Mục tiêu của ta là tìm giá trị x nhỏ nhất sao cho số phân đoạn cần thiết không vượt quá k.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Greedy Check) Với một mức dung lượng giới hạn là x, ta sẽ dùng thuật toán Tham lam để chia đoạn. Duyệt qua từng gói tin, ta cố gắng nhồi nhét nhiều gói tin nhất có thể vào đoạn hiện tại sao cho tổng không vượt quá x. Nếu thêm gói tin tiếp theo mà bị quá tải, ta bắt buộc phải đóng đoạn hiện tại và chuyển gói tin đó sang đoạn mới.

```cpp
int n, k;
int a[100005];

bool check(long long x){
    int cnt = 1;
    long long cur_sum = 0;

    for (int i = 1; i <= n; i++){
        if (cur_sum + a[i] > x){
            cnt++;
            cur_sum = a[i];
        }
        else{
            cur_sum += a[i];
        }
    }
    return cnt <= k;
}
```

Bước 2: Tìm kiếm nhị phân giá trị x Việc xác định vùng tìm kiếm $[L, R]$ rất quan trọng để thuật toán chạy đúng và tối ưu hiệu suất:

- **Cận dưới (L):** Dung lượng của một đoạn không thể nhỏ hơn gói tin lớn nhất bên trong nó. Do đó, L chính là giá trị lớn nhất trong mảng A.
- **Cận trên (R):** Trường hợp xấu nhất ($k = 1$), ta không cắt đoạn nào cả, khi đó dung lượng lớn nhất chính là tổng toàn bộ mảng A.

```cpp
void solve(){
    long long left = 0, right = 0;
    for (int i = 1; i <= n; i++){
        left = max(left, (long long)a[i]);
        right += a[i];
    }

    long long ans = right;

    while (left <= right){
        long long mid = (right + left) / 2;
        if (check(mid)){
            ans = mid;
            right = mid - 1;
        }
        else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Gộp các thành phần lại, ta có giải pháp với độ phức tạp $O(n \log(\sum A_i))$, cực kỳ tối ưu và tinh gọn.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, k;
int a[100005];

bool check(long long x){
    int cnt = 1;
    long long cur_sum = 0;

    for (int i = 1; i <= n; i++){
        if (cur_sum + a[i] > x){
            cnt++;
            cur_sum = a[i];
        }
        else{
            cur_sum += a[i];
        }
    }
    return cnt <= k;
}

void solve(){
    long long left = 0, right = 0;
    for (int i = 1; i <= n; i++){
        left = max(left, (long long)a[i]);
        right += a[i];
    }

    long long ans = right;

    while (left <= right) {
        long long mid = left + (right - left) / 2;
        if (check(mid)){
            ans = mid;
            right = mid - 1;
        }
        else{
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}

signed main(){
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    cin >> n >> k;
    for (int i = 1; i <= n; i++){
        cin >> a[i];
    }

    solve();

    return 0;
}
```

Bài 10: Bảng cửu chương (CSES - Multiplication Table)

1. Phân tích bài toán

- Bài toán yêu cầu tìm phần tử ở vị trí chính giữa khi sắp xếp bảng cửu chương kích thước $n \times n$. Vì n là số lẻ, tổng số phần tử của bảng là n^2 (cũng là số lẻ).
- Vị trí chính giữa (trung vị - median) của dãy số đã sắp xếp chính là phần tử thứ $\frac{n^2}{2} + 1$ (chia lấy phần nguyên).
- **Khó khăn:** Với $n \leq 10^6$, tổng số phần tử n^2 có thể lên tới 10^{12}. Ta hoàn toàn không thể tạo mảng để lưu và sắp xếp 10^{12} con số này được (chắc chắn sẽ bị MLE và TLE).
- **Giải pháp:** Tìm kiếm nhị phân kết quả (Binary Search on Answer). Ta cần tìm giá trị x nhỏ nhất sao cho số lượng phần tử trong bảng cửu chương **nhỏ hơn hoặc bằng** x đạt ít nhất $\frac{n^2}{2} + 1$.

2. Xây dựng từng bước

Bước 1: Hàm đếm số phần tử $\leq x$ Bảng cửu chương được chia thành n hàng. Hàng thứ i là các bội số của i, lần lượt là: $i, 2i, 3i, \ldots, ni$. Với một giá trị x cho trước, số lượng phần tử $\leq x$ nằm trên hàng thứ i sẽ là $\lfloor \frac{x}{i} \rfloor$. Tuy nhiên, mỗi hàng chỉ có tối đa n phần tử, nên ta phải lấy giá trị nhỏ hơn giữa $\frac{x}{i}$ và n. Duyệt qua n hàng, ta có tổng số phần tử $\leq x$.

```cpp
int n;

bool check(int x) {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        cnt += min(x / i, n);
    }
    int target = (n * n) / 2 + 1;
    return cnt >= target;
}
```

Bước 2: Tìm kiếm nhị phân giá trị x Phạm vi của kết quả nằm trong khoảng từ phần tử nhỏ nhất của bảng là 1 (ở ô 1×1), cho đến phần tử lớn nhất là n^2 (ở ô $n \times n$). Ta thực hiện chặt nhị phân trong đoạn $[1, n^2]$. Nếu hàm `check(mid)` trả về đúng (tức là đã có đủ ít nhất một nửa số phần tử $\leq mid$), ta lưu lại kết quả và ép giới hạn trên `right = mid - 1` để cố gắng tìm một giá trị x nhỏ hơn nữa.

```cpp
void solve() {
    int left = 1, right = n * n;
    int ans = right;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Chú ý: Do $n \le 10^6$, tích $n \times n$ sẽ lên tới 10^{12}, vượt quá giới hạn của số nguyên 32-bit. Ta bắt buộc phải `define int long long` để tránh lỗi tràn số (Overflow). Độ phức tạp của thuật toán là $O(N \log(N^2))$, chạy rất thoải mái trong 1.0s.

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;

bool check(int x) {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        cnt += min(x / i, n);
    }
    int target = (n * n) / 2 + 1;
    return cnt >= target;
}

void solve() {
    int left = 1, right = n * n;
    int ans = right;

    while (left <= right) {
        int mid = left + (right - left) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n) {
        solve();
    }

    return 0;
}
```

Bài 11: Dãy con có trung bình cộng lớn nhất (Maximum Average Subarray)

1. Phân tích bài toán

- **Biến đổi toán học:** Bài toán yêu cầu tìm một dãy con từ L đến R (với $R - L + 1 \ge k$) sao cho trung bình cộng lớn nhất. Giả sử ta muốn kiểm tra xem có tồn tại dãy con nào đạt trung bình cộng lớn hơn hoặc bằng một giá trị x hay không:

$$\frac{A_L + A_{L+1} + \cdots + A_R}{R - L + 1} \ge x$$

$$\Leftrightarrow (A_L - x) + (A_{L+1} - x) + \cdots + (A_R - x) \ge 0$$

- Nhờ phép biến đổi này, ta đã chuyển bài toán từ "tính trung bình cộng" (phức tạp vì độ dài dãy thay đổi) sang bài toán "tìm dãy con có tổng ≥ 0" trên một mảng mới mà mỗi phần tử đã bị trừ đi x.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Sử dụng Mảng cộng dồn - Prefix Sum) Với giá trị x cho trước, ta tạo mảng cộng dồn B sao cho $B[i] = B[i-1] + (A[i] - x)$. Tổng của dãy con từ vị trí $j+1$ đến i sẽ là $B[i] - B[j]$. Để dãy con này có độ dài ít nhất k, ta phải có $i - j \ge k$, hay $j \le i - k$. Muốn tổng $B[i] - B[j] \ge 0$ đạt khả thi cao nhất, ta cần trừ đi một lượng $B[j]$ càng nhỏ càng tốt. Vậy khi duyệt i, ta chỉ cần duy trì giá trị nhỏ nhất của mảng cộng dồn từ đầu đến vị trí $i - k$.

```cpp
int n, k;
double a[100005];
double b[100005];

bool check(double x) {
    double min_val = 0.0;
    for (int i = 1; i <= n; i++) {
        b[i] = b[i - 1] + (a[i] - x);
        if (i >= k) {
            min_val = min(min_val, b[i - k]);
            if (b[i] >= min_val) {
                return true;
            }
        }
    }
    return false;
}
```

Bước 2: Tìm kiếm nhị phân trên số thực Do giới hạn $1 \le A_i \le 10^9$, trung bình cộng chắc chắn nằm trong khoảng từ 0 đến 10^9. Vì đây là số thực, ta không thể dùng ‘mid + 1‘ hay ‘mid - 1‘ như số nguyên. Đồng thời, để tránh lỗi sai số (Precision Error) dễ gây lặp vô hạn nếu dùng ‘while (r - l >= epsilon)‘, ta sẽ dùng vòng lặp ‘for‘ chạy cố định 100 lần. Sau 100 lần chia đôi, khoảng cách sẽ thu hẹp lại cực kỳ nhỏ, thừa sức đáp ứng độ chính xác 10^{-3} của đề bài.

```cpp
void solve() {
    double left = 0.0, right = 1e9;
    double ans = 0.0;
    for (int iter = 0; iter < 100; iter++) {
        double mid = (left + right) / 2.0;
        if (check(mid)) {
            ans = mid;
            left = mid;
        } else {
            right = mid;
        }
    }
    cout << fixed << setprecision(3) << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Mảng `a` và `b` dùng kiểu `double` có thể biểu diễn an toàn các con số lên tới 10^{15} mà không bị mất mát dữ liệu, nên với $n = 10^5$ và $A_i = 10^9$, mảng cộng dồn của ta không lo bị tràn hay sai số.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, k;
double a[100005];
double b[100005];

bool check(double x) {
    double min_val = 0.0;
    for (int i = 1; i <= n; i++) {
        b[i] = b[i - 1] + (a[i] - x);
        if (i >= k) {
            min_val = min(min_val, b[i - k]);
            if (b[i] >= min_val) {
                return true;
            }
        }
    }
    return false;
}

void solve() {
    double left = 0.0, right = 1e9;
    double ans = 0.0;
    for (int iter = 0; iter < 100; iter++) {
        double mid = (left + right) / 2.0;
        if (check(mid)) {
            ans = mid;
            left = mid;
        } else {
            right = mid;
        }
    }

    cout << fixed << setprecision(3) << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> k) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }
        solve();
    }

    return 0;
}
```

Bài 12: Hàng rào năng lượng (FENCE2)

1. Phân tích bài toán

- **Dấu hiệu:** Đề bài yêu cầu tìm cách thiết lập sao cho "mức năng lượng của phân đoạn yếu nhất là lớn nhất". Đây chính là câu thần chú quen thuộc của kỹ thuật Tìm kiếm nhị phân trên kết quả.
- **Tính đơn điệu:** Giả sử ta muốn nâng mức năng lượng tối thiểu của toàn bộ hàng rào lên một mức x. Nếu x càng cao, ta càng phải sử dụng nhiều lõi pin xịn (và phải vứt bỏ nhiều lõi pin yếu). Đến một mức x quá lớn, ta sẽ cạn kiệt pin dự phòng. Ngược lại, mức x nhỏ thì rất dễ đáp ứng.
- **Ràng buộc di chuyển:** Leon chỉ có thể tiến tới, không được lùi và không thể nhặt lại pin. Điều này gợi ý cho ta sử dụng thuật toán Tham lam (Greedy kết hợp kỹ thuật Hai con trỏ - Two Pointers) để mô phỏng quá trình thử pin.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Mô phỏng bằng Tham lam & Hai con trỏ) Ta giả sử cần kiểm tra xem có thể nâng tất cả các đoạn rào lên mức năng lượng $\ge x$ hay không. Ta dùng một vòng lặp duyệt qua các đoạn rào, và một con trỏ `id` để duyệt qua các lõi pin. Nếu đoạn rào hiện tại đã $\ge x$, ta bỏ qua. Nếu chưa đủ, ta liên tục lướt qua (vứt bỏ) các lõi pin không đủ sức bù đắp (`a[i] + b[id] < x`). Ta dừng lại khi tìm được lõi pin đầu tiên thỏa mãn và gắn nó vào đoạn rào (`id++`). Nếu duyệt hết pin mà vẫn không cứu được đoạn rào, nghĩa là mức x này là bất khả thi.

```cpp
int n, m;
int a[100005], b[100005];

bool check(int x) {
    int id = 1;
    for (int i = 1; i <= n; i++) {
        if (a[i] >= x) {
            continue;
        }

        while (id <= m && a[i] + b[id] < x) {
            id++;
        }

        if (id > m) {
            return false;
        }
        id++;
    }
    return true;
}
```

Bước 2: Tìm kiếm nhị phân Mức năng lượng thấp nhất có thể đạt được là 1. Mức năng lượng lớn nhất có thể đạt được là khi một đoạn rào lấy được lõi pin mạnh nhất. Với $a_i, b_j \le 10^8$, mức năng lượng tối đa sau khi cộng không vượt quá $2 \cdot 10^8$. Do đó, ta có thể an tâm đặt cận trên `right = 1e9`.

```cpp
void solve() {
    int left = 1, right = 1e9, ans = 0;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Độ phức tạp của thuật toán kiểm tra là $O(n + m)$ vì con trỏ đoạn rào và con trỏ pin đều chỉ tiến lên chứ không quay lại. Kết hợp với tìm kiếm nhị phân, tổng độ phức tạp là $O((n + m)\log(\max(A_i + B_i)))$, chạy cực kỳ nhẹ nhàng trong giới hạn thời gian.

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
int a[100005], b[100005];

bool check(int x) {
    int id = 1;
    for (int i = 1; i <= n; i++) {
        if (a[i] >= x) {
            continue;
        }

        while (id <= m && a[i] + b[id] < x) {
            id++;
        }

        if (id > m) {
            return false;
        }
        id++;
    }
    return true;
}

void solve() {
    int left = 1, right = 1e9, ans = 0;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }
        cin >> m;
        for (int i = 1; i <= m; i++) {
            cin >> b[i];
        }

        solve();
    }

    return 0;
}
```

Bài 13: Xưởng sản xuất đèn lồng (LANTERN)

1. Phân tích bài toán

- **Dấu hiệu:** Đề bài yêu cầu tìm "thời gian hoàn thành ít nhất có thể". Vì các robot làm việc song song, thời gian hoàn thành toàn bộ chính là thời gian của robot làm việc lâu nhất. Đây là dạng toán "Minimize the Maximum", dấu hiệu đặc trưng của kỹ thuật Tìm kiếm nhị phân trên kết quả.
- **Tính đơn điệu:** Gọi x là thời gian tối đa cho phép để hoàn thành công việc. Nếu x càng lớn, mỗi robot sẽ làm được nhiều lồng đèn hơn, do đó ta cần ít robot hơn. Ngược lại, nếu ép x nhỏ lại, ta sẽ cần huy động rất nhiều robot. Ta cần tìm x nhỏ nhất sao cho tổng số robot cần dùng không vượt quá n.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Check) Với quỹ thời gian là x, một robot chỉ có thể làm tối đa x chiếc lồng đèn cùng loại. Để sản xuất đủ a_i chiếc lồng đèn loại i, số lượng robot cần phân công sẽ là $\lceil \frac{a_i}{x} \rceil$. Trong C++, ta có thể tính nhanh phép chia lấy trần này bằng công thức `a[i] / x + (a[i] % x != 0)` hoặc `(a[i] + x - 1) / x`. Cộng dồn số robot cần thiết cho toàn bộ m loại lồng đèn, nếu tổng số này $\le n$, mức thời gian x là khả thi.

```cpp
long long n, m;
long long a[100005];

bool check(long long x) {
    long long cnt = 0;
    for (int i = 1; i <= m; i++) {
        cnt += a[i] / x + (a[i] % x != 0);
    }
    return cnt <= n;
}
```

Bước 2: Tìm kiếm nhị phân kết quả Thời gian tối thiểu (cận dưới) là 1 (vì $a_i \ge 1$). Thời gian tối đa (cận trên) là khi ta chỉ có 1 robot để làm loại lồng đèn có số lượng lớn nhất, tức là 10^9. Ta sẽ chặt nhị phân trong khoảng $[1, 10^9]$.

```cpp
void solve() {
    long long left = 1, right = 1e9, ans = right;

    while (left <= right) {
        long long mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Lưu ý quan trọng: Biến `cnt` trong hàm `check` bắt buộc phải dùng kiểu `long long`. Trong trường hợp $x = 1$, `cnt` sẽ bằng tổng toàn bộ mảng A, có thể đạt tới 10^{14} và gây tràn kiểu `int` 32-bit.

```cpp
#include <bits/stdc++.h>
using namespace std;

long long n, m;
long long a[100005];

bool check(long long x) {
    long long cnt = 0;
    for (int i = 1; i <= m; i++) {
        cnt += a[i] / x + (a[i] % x != 0);
    }
    return cnt <= n;
}

void solve() {
    long long left = 1, right = 1e9, ans = right;

    while (left <= right) {
        long long mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> m) {
        for (int i = 1; i <= m; i++) {
            cin >> a[i];
        }
        solve();
    }

    return 0;
}
```

Bài 14: Dung hợp Lõi Năng Lượng

1. Phân tích bài toán

- **Bản chất thao tác:** Khi lõi a_i hấp thụ lõi a_j ($a_i \ge a_j$), lõi a_j biến mất và a_i tăng thêm 1 đơn vị. Nói cách khác, để tăng mức năng lượng của một lõi lên 1, ta phải "hi sinh" một lõi khác có năng lượng nhỏ hơn hoặc bằng nó.
- **Mục tiêu:** Tối đa hóa số lượng lõi đạt mức năng lượng $\ge k$.
- **Chiến thuật Tham lam (Greedy):**
  - Những lõi đã $\ge k$ thì ta giữ nguyên, không cần nâng cấp.
  - Để tối ưu số lõi đạt chuẩn, ta nên ưu tiên nâng cấp những lõi có năng lượng đang ở gần mức k nhất (vì chúng cần ít vật tế nhất).

  - Những lõi được đem đi làm vật hi sinh (vật tế) nên là những lõi có mức năng lượng thấp nhất trong mảng.
- **Tính đơn điệu:** Giả sử ta muốn nâng cấp các lõi từ vị trí *mid* đến vị trí *pos* (với *pos* là lõi lớn nhất đang $< k$). Nếu ta mở rộng khoảng này (tức là giảm chỉ số *mid* xuống), ta sẽ cần nhiều năng lượng bù đắp hơn, nhưng lại có ít lõi dư thừa để hi sinh hơn. Sự đối nghịch này tạo ra một hàm đơn điệu, cho phép ta dùng Tìm kiếm nhị phân để tìm chỉ số *mid* nhỏ nhất có thể.

2. Xây dựng từng bước

Bước 1: Chuẩn bị dữ liệu Sắp xếp mảng lõi năng lượng tăng dần. Sau đó, xây dựng Mảng cộng dồn `pre` để có thể tính nhanh tổng năng lượng của một đoạn bất kỳ trong $O(1)$.

```cpp
// Do k, a[i] len den 10^9 nen viec tinh tong cong don hoac tinh so luong can bu dap
// rat de vuot qua gioi han int. Ta nen su dung long long cho toan bo bai toan.
int n, q;
int arr[100005], pre[100005];

// Trong ham main:
// sort(arr + 1, arr + n + 1);
// for (int i = 1; i <= n; i++) {
//     pre[i] = pre[i - 1] + arr[i];
// }
```

Bước 2: Xử lý từng truy vấn với Tìm kiếm nhị phân Với mỗi kịch bản k:

- Tìm vị trí *pos* của phần tử lớn nhất $< k$. Các phần tử từ $pos + 1$ đến n đã đạt chuẩn k, ta có sẵn $n - pos$ lõi.
- Ta thử nâng cấp các phần tử từ chỉ số *mid* đến *pos*. Số lượng thao tác cần thiết để đưa toàn bộ dải này lên mức k là:

$$\text{Cần} = k \times (pos - mid + 1) - \sum_{i=mid}^{pos} arr[i]$$

- Số lượng lõi có thể dùng làm vật hi sinh chính là số lõi nằm trước *mid*, tức là $mid - 1$ lõi.
- Nếu $mid - 1 \geq$ Cần, phương án nâng cấp này là khả thi. Ta lưu lại kết quả và thử mở rộng dải nâng cấp bằng cách ép `right = mid - 1`. Ngược lại, ép `left = mid + 1`.

```cpp
void solve(int k) {
    int pos = lower_bound(arr + 1, arr + n + 1, k) - arr - 1;
    int left = 1, right = pos, ans = pos + 1;

    while (left <= right) {
        int mid = (left + right) / 2;
        int required = k * (pos - mid + 1) - (pre[pos] - pre[mid - 1])

        if (mid - 1 >= required) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << (n - pos) + (pos - ans + 1) << '\n';
}
```

3. Mã nguồn hoàn chỉnh Độ phức tạp của thuật toán cho mỗi truy vấn chỉ là $O(\log N)$, do đó tổng thời gian cho toàn bộ Q truy vấn là $O(N \log N + Q \log N)$, chạy cực mượt trong 1.0s.

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int arr[100005], pre[100005];
int n, q;

void solve(int k) {
    int pos = lower_bound(arr + 1, arr + n + 1, k) - arr - 1;
    int left = 1, right = pos, ans = pos + 1;

    while (left <= right) {
        int mid = (left + right) / 2;
        int required = k*(pos - mid + 1) - (pre[pos] - pre[mid - 1]);

        if (mid - 1 >= required) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << (n - pos) + (pos - ans + 1) << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    cin >> n >> q;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    sort(arr + 1, arr + n + 1);

    for (int i = 1; i <= n; i++) {
        pre[i] = pre[i - 1] + arr[i];
    }

    while (q--) {
        int k;
        cin >> k;
        solve(k);
    }
    return 0;
}
```

Bài 15: Phân bổ năng lượng 3 pha

1. Phân tích bài toán

- **Khó khăn:** Bài toán yêu cầu chia mảng thành 3 đoạn liên tiếp. Nếu dùng hai vòng lặp lồng nhau để thử mọi vị trí cắt, độ phức tạp sẽ là $O(N^2)$, với $N = 10^5$ chắc chắn sẽ bị quá thời gian (TLE).
- **Ý tưởng tối ưu:** Thay vì tìm cả 2 vị trí cắt cùng lúc, ta sẽ **cố định vị trí cắt đầu tiên** (gọi là i). Khi đó:
  - Pha 1 đã được xác định: là đoạn từ 1 đến i.
  - Tổng năng lượng còn lại cần chia cho Pha 2 và Pha 3 là: $\text{sum\_rem} = A[N] - A[i]$.
- Để độ chênh lệch giữa 3 pha là nhỏ nhất, lý tưởng nhất là Pha 2 và Pha 3 nên cân bằng nhau. Tức là tổng năng lượng của Pha 2 cần xấp xỉ $\text{sum\_rem}/2$.
- Vì các khối năng lượng đều dương, mảng cộng dồn luôn tăng ngặt. Ta có thể dùng Tìm kiếm nhị phân (`lower_bound`) để tìm vị trí cắt thứ hai (gọi là j) sao cho tổng đoạn từ 1 đến j gần với $A[i] + \text{sum\_rem}/2$ nhất.

2. Xây dựng từng bước

Bước 1: Mảng cộng dồn (Prefix Sum) Tính mảng cộng dồn A để có thể truy vấn tổng năng lượng của một đoạn bất kỳ từ L đến R trong thời gian $O(1)$.

$$\text{get}(L, R) = A[R] - A[L-1]$$

Bước 2: Cố định điểm cắt 1, tìm kiếm điểm cắt 2 Duyệt i từ 1 đến $N - 2$ (để chừa ít nhất 2 khối cho Pha 2 và Pha 3). Với mỗi i:

- Tìm mục tiêu lý tưởng cho điểm cắt j: $\text{target} = A[i] + (A[N] - A[i])/2$.
- Dùng `lower_bound` tìm vị trí *pos* đầu tiên $\geq$ target.
- Vì vị trí lý tưởng có thể nằm ngay trước hoặc ngay tại *pos*, ta phải thử tính toán độ chênh lệch $\max(x, y, z) - \min(x, y, z)$ ở cả 2 trường hợp cắt tại *pos* và $pos - 1$, sau đó cập nhật kết quả nhỏ nhất.

```cpp
// Doan logic chinh (nam trong ham main)
long long ans = 1e18;

// i la diem ket thuc cua Pha 1
for (int i = 1; i <= n - 2; i++) {
    long long sum_rem = a[n] - a[i];
    long long target = a[i] + sum_rem / 2;

    // Tim diem ket thuc cua Pha 2
    int pos = lower_bound(a + i + 1, a + n, target) - a;

    // Thu cat tai pos (dam bao pos < n de Pha 3 co it nhat 1 khoi)
    if (pos < n) {
        long long x = a[i];
        long long y = get_sum(i + 1, pos);
        long long z = get_sum(pos + 1, n);
        ans = min(ans, max({x, y, z}) - min({x, y, z}));
    }

    // Thu cat tai pos - 1 (dam bao pos - 1 > i de Pha 2 co it nhat 1 khoi)
    if (pos - 1 > i) {
        long long x = a[i];
        long long y = get_sum(i + 1, pos - 1);
        long long z = get_sum(pos - 1 + 1, n);
        ans = min(ans, max({x, y, z}) - min({x, y, z}));
    }
}
```

3. Mã nguồn hoàn chỉnh Độ phức tạp của thuật toán giảm xuống chỉ còn $O(N \log N)$, dư sức vượt qua giới hạn 1.0s. Dù mảng không quá lớn, ta vẫn nên khai báo mảng cộng dồn kiểu `long long` để luyện thói quen an toàn, chống tràn số.

```cpp
#include <bits/stdc++.h>
using namespace std;

long long a[100005];

long long get_sum(int l, int r) {
    return a[r] - a[l - 1];
}

int main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    int n;
    if (cin >> n) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
            a[i] += a[i - 1];
        }

        long long ans = 1e18;

        for (int i = 1; i <= n - 2; i++) {
            long long sum_rem = a[n] - a[i];
            long long target = a[i] + sum_rem / 2;

            int pos = lower_bound(a + i + 1, a + n, target) - a;

            if (pos < n) {
                long long x = a[i];
                long long y = get_sum(i + 1, pos);
                long long z = get_sum(pos + 1, n);
                ans = min(ans, max({x, y, z}) - min({x, y, z}));
            }

            if (pos - 1 > i) {
                long long x = a[i];
                long long y = get_sum(i + 1, pos - 1);
                long long z = get_sum(pos - 1 + 1, n);
                ans = min(ans, max({x, y, z}) - min({x, y, z}));
            }
        }

        cout << ans << '\n';
    }

    return 0;
}
```

Bài 16: Khai thác tinh thể (SAW - Chặt Cây)

1. Phân tích bài toán

- **Dấu hiệu:** Đề yêu cầu tìm "độ cao H lớn nhất có thể thiết lập sao cho... thu được ít nhất M". Đây là dạng toán Tìm giá trị lớn nhất thỏa mãn một điều kiện cho trước, đặc trưng của Tìm kiếm nhị phân trên kết quả.
- **Tính đơn điệu:** Giả sử ta thiết lập lưới laser ở độ cao H. Nếu H càng lớn, phần ngọn tinh thể bị cắt càng ngắn, lượng thu thập được càng ít. Ngược lại, nếu H càng nhỏ (cắt càng sâu xuống gốc), lượng tinh thể thu được càng nhiều. Ta cần tìm mức H cao nhất nhưng vẫn đảm bảo tổng lượng cắt được $\geq M$.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Check) Với một độ cao H cố định, ta duyệt qua toàn bộ N trụ tinh thể. Nếu trụ nào cao hơn H, ta cộng phần chênh lệch $(A_i - H)$ vào tổng sản lượng. Cuối cùng, so sánh xem tổng này có đạt mức M yêu cầu hay không.

```cpp
int n, m;
int a[1000005];

bool check(int h) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] > h) {
            sum += (a[i] - h);
        }
    }
    return sum >= m;
}
```

Bước 2: Tìm kiếm nhị phân giá trị H Độ cao thấp nhất có thể cắt là 0 (cắt tận gốc toàn bộ khu rừng). Độ cao lớn nhất có thể cắt là chiều cao của trụ tinh thể cao nhất trong rừng. Ta tiến hành chặt nhị phân trong khoảng này. Nếu tại độ cao `mid` mà vẫn thu đủ tinh thể, ta lưu lại kết quả và thử nâng `left = mid + 1` để xem có thể cắt cao hơn được nữa không (nhằm tiết kiệm năng lượng). Nếu không đủ, ta bắt buộc phải hạ lưỡi cắt xuống bằng cách ép `right = mid - 1`.

```cpp
void solve() {
    int left = 0, right = 0;
    for (int i = 1; i <= n; i++) {
        right = max(right, a[i]);
    }

    int ans = 0;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Lưu ý: Mức yêu cầu M lên tới $2 \cdot 10^9$ và tổng lượng tinh thể cắt được có thể còn lớn hơn thế rất nhiều. Do đó, biến `sum` và các biến liên quan bắt buộc phải dùng kiểu `long long` (được `define` thành `int`) để tránh tràn số. Giới hạn $N \leq 10^6$, mảng tốn khoảng 8MB, hoàn toàn an toàn trong giới hạn bộ nhớ 256MB.

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int a[1000005];

bool check(int h) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        if (a[i] > h) {
            sum += (a[i] - h);
        }
    }
    return sum >= m;
}

void solve() {
    int left = 0, right = 0;
    for (int i = 1; i <= n; i++) {
        right = max(right, a[i]);
    }

    int ans = 0;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> m) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }
        solve();
    }

    return 0;
}
```

Bài 17: Quét phổ tần số (SUBMATRIX - Ma trận con)

1. Phân tích bài toán

- **Dấu hiệu:** Đề bài yêu cầu tìm "độ dài cạnh lớn nhất có thể của ma trận vuông con" thỏa mãn điều kiện tần số khác nhau $\leq K$. Đây tiếp tục là dạng Tìm kiếm nhị phân trên tập kết quả.
- **Tính đơn điệu:** Gọi x là độ dài cạnh của vùng an toàn. Nếu x càng lớn, diện tích ma trận con càng to, nó sẽ bao trọn càng nhiều giá trị khác nhau. Ngược lại, x nhỏ thì số lượng giá trị phân biệt sẽ ít đi. Do đó, hàm số lượng giá trị phân biệt theo x là một hàm đồng biến. Ta có thể tìm kiếm nhị phân giá trị x.

2. Xây dựng từng bước

Bước 1: Kỹ thuật Cửa sổ trượt 2D (2D Sliding Window) Giả sử ta cần kiểm tra xem có ma trận vuông kích thước $x \times x$ nào thỏa mãn số lượng phần tử khác nhau $\leq K$ hay không. Nếu kiểm tra "trâu" (naive) từng ma trận con, ta sẽ mất $O(N^4)$, chắc chắn bị Quá thời gian (TLE).

Để tối ưu, ta sẽ trượt một khung cửa sổ kích thước $x \times x$ theo từng hàng từ trái sang phải:

- Khi bắt đầu một hàng mới, ta dựng khung cửa sổ ở vị trí ngoài cùng bên trái.
- Khi trượt khung sang phải 1 ô, ta không cần đếm lại từ đầu. Ta chỉ việc **xóa đi cột ngoài cùng bên trái** và **thêm vào cột mới ở bên phải**.
- Kỹ thuật này giảm thời gian kiểm tra một khung hình xuống chỉ còn $O(x)$ thay vì $O(x^2)$.

```cpp
int m, n, k;
int a[305][305];
int cnt[90005];
int dif_num;

void add_val(int val) {
    if (cnt[val] == 0) dif_num++;
    cnt[val]++;
}

void del_val(int val) {
    cnt[val]--;
    if (cnt[val] == 0) dif_num--;
}

bool check(int x) {
    // Duyet tung hang ngang
    for (int i = 1; i <= m - x + 1; i++) {
        dif_num = 0;
        memset(cnt, 0, sizeof(cnt)); // Lam sach mang dem cho hang moi

        // Dung khung cua so dau tien o mep ben trai
        for (int r = i; r < i + x; r++) {
            for (int c = 1; c <= x; c++) {
                add_val(a[r][c]);
            }
        }
        if (dif_num <= k) return true;

        // Truot khung sang phai tung o 1
        for (int j = 2; j <= n - x + 1; j++) {
            // Go bo cot ben trai
            for (int r = i; r < i + x; r++) {
                del_val(a[r][j - 1]);
            }
            // Them cot ben phai vao
            for (int r = i; r < i + x; r++) {
                add_val(a[r][j + x - 1]);
            }

            if (dif_num <= k) return true;
        }
    }
    return false;
}
```

Bước 2: Tìm kiếm nhị phân kích thước x Kích thước nhỏ nhất của ma trận con là 1. Kích thước lớn nhất là $\min(M, N)$. Ta chặt nhị phân để tìm giá trị thỏa mãn lớn nhất.

```
void solve() {
    int left = 1, right = min(m, n);
    int ans = 0;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Tổng độ phức tạp của thuật toán là $O(\log(\min(M, N)) \cdot M \cdot N \cdot x) \approx O(N^3 \log N)$. Với $N \leq 300$, thuật toán xử lý mất khoảng chục triệu phép toán, chạy dư sức trong 1.0s.

```
#include <bits/stdc++.h>
using namespace std;

int m, n, k;
int a[305][305];
int cnt[90005];
int dif_num;

void add_val(int val) {
    if (cnt[val] == 0) dif_num++;
    cnt[val]++;
}

void del_val(int val) {
    cnt[val]--;
    if (cnt[val] == 0) dif_num--;
}

bool check(int x) {
    for (int i = 1; i <= m - x + 1; i++) {
        dif_num = 0;
        memset(cnt, 0, sizeof(cnt));

        for (int r = i; r < i + x; r++) {
            for (int c = 1; c <= x; c++) {
                add_val(a[r][c]);
            }
        }
        if (dif_num <= k) {
            return true;
        }

        for (int j = 2; j <= n - x + 1; j++) {
            for (int r = i; r < i + x; r++) {
                del_val(a[r][j - 1]);
            }

            for (int r = i; r < i + x; r++) {
                add_val(a[r][j + x - 1]);
            }

            if (dif_num <= k) {
                return true;
            }
        }
    }
    return false;
}

void solve() {
    int left = 1, right = min(m, n);
    int ans = 0;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> m >> n >> k) {
        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                cin >> a[i][j];
            }
        }
        solve();
    }

    return 0;
}
```

Bài 18: Đối chiếu tín hiệu lượng tử

1. Phân tích bài toán

- **Cạm bẫy độ phức tạp:** Số lượng dải con liên tiếp của chuỗi Alpha độ dài m là $\frac{m(m+1)}{2} \approx 5 \cdot 10^5$. Tương tự, chuỗi Beta cũng có khoảng $5 \cdot 10^5$ dải con. Nếu dùng 4 vòng lặp duyệt mọi cặp dải con, số phép toán là $(5 \cdot 10^5) \times (5 \cdot 10^5) = 2.5 \cdot 10^{11}$ → Chắc chắn bị Quá thời gian (TLE).
- **Hướng giải quyết (Sinh mảng và Chặt nhị phân):**
  - Ta sẽ tính trước tổng của **tất cả** các dải con trong chuỗi Alpha và lưu vào một mảng `sumA`.
  - Làm tương tự với chuỗi Beta, lưu vào mảng `sumB`.
  - Sắp xếp mảng `sumB` tăng dần.
  - Với mỗi giá trị x trong `sumA`, ta dùng Tìm kiếm nhị phân (`lower_bound` và `upper_bound`) trên `sumB` để đếm nhanh số lượng phần tử nhỏ hơn, bằng, và lớn hơn x.

2. Xây dựng từng bước

Bước 1: Mảng cộng dồn & Sinh dải con Tính mảng cộng dồn cho cả Alpha và Beta để có thể lấy tổng một dải con bất kỳ trong $O(1)$. Sau đó, dùng 2 vòng lặp lồng nhau cho mỗi chuỗi để sinh ra toàn bộ giá trị tổng và đẩy vào `vector`.

```
// Khai bao trong ham main
for (int i = 1; i <= m; i++) {
    for (int j = i; j <= m; j++) {
        sumA.push_back(preA[j] - preA[i - 1]);
    }
}

for (int i = 1; i <= n; i++) {
    for (int j = i; j <= n; j++) {
        sumB.push_back(preB[j] - preB[i - 1]);
    }
}
```

Bước 2: Tìm kiếm nhị phân đếm số lượng Sau khi sắp xếp `sumB`, với mỗi tổng x thuộc `sumA`:

- Số lượng phần tử trong Beta $< x$ (Alpha áp đảo - Tập G): Chính là khoảng cách từ đầu mảng `sumB` đến `lower_bound(x)`.
- Số lượng phần tử trong Beta $= x$ (Cộng hưởng - Tập E): Chính là khoảng cách từ `lower_bound(x)` đến `upper_bound(x)`.
- Số lượng phần tử trong Beta $> x$ (Alpha nhỏ hơn - Tập S): Chính là phần còn lại, từ `upper_bound(x)` đến cuối mảng `sumB`.

3. Mã nguồn hoàn chỉnh Lưu ý: Mức năng lượng a_i, b_j có thể lên đến 10^9, tổng của 1000 phần tử sẽ lên đến 10^{12}, do đó các mảng tổng bắt buộc phải dùng kiểu `long long`. Hơn nữa, kết quả số lượng phần tử của S, E, G cũng có thể đạt $2.5 \cdot 10^{11}$, nên ba biến đếm cũng phải dùng `long long`. Độ phức tạp thuật toán là $O(m^2 + n^2 \log(n^2))$, chạy rất mượt mà.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int m, n;
int a[1005], b[1005];
int preA[1005], preB[1005];
vector<int> sumA, sumB;

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> m >> n) {
        for (int i = 1; i <= m; i++) {
            cin >> a[i];
            preA[i] = preA[i - 1] + a[i];
        }

        for (int i = 1; i <= n; i++) {
            cin >> b[i];
            preB[i] = preB[i - 1] + b[i];
        }

        for (int i = 1; i <= m; i++) {
            for (int j = i; j <= m; j++) {
                sumA.push_back(preA[j] - preA[i - 1]);
            }
        }

        for (int i = 1; i <= n; i++) {
            for (int j = i; j <= n; j++) {
                sumB.push_back(preB[j] - preB[i - 1]);
            }
        }

        sort(sumB.begin(), sumB.end());

        int S = 0, E = 0, G = 0;

        for (int x : sumA) {
            auto it1 = lower_bound(sumB.begin(), sumB.end(), x);
            auto it2 = upper_bound(sumB.begin(), sumB.end(), x);

            G += (it1 - sumB.begin());
            E += (it2 - it1);
            S += (sumB.end() - it2);
        }

        cout << S << "␣" << E << "␣" << G << '\n';
    }

    return 0;
}
```

Bài 19: Dây chuyền cơ giáp (Factory Machines / Gà đẻ trứng)

1. Phân tích bài toán

- **Dấu hiệu:** Đề yêu cầu tìm "thời gian ngắn nhất... để sản xuất đủ". Đây là một bài toán kinh điển của dạng Tìm kiếm nhị phân trên kết quả.
- **Tính đơn điệu:** Gọi x là thời gian xưởng hoạt động. Rõ ràng nếu x càng lớn, số lượng cơ giáp in ra được càng nhiều. Ngược lại, thời gian càng ngắn thì sản lượng càng thấp. Hàm sản lượng theo thời gian là một hàm đồng biến, do đó ta có thể tìm kiếm nhị phân giá trị x.

2. Xây dựng từng bước

Bước 1: Hàm kiểm tra (Check) Với một mức thời gian là x (giây), máy thứ i (cần T_i giây/sản phẩm) sẽ in được chính xác $\lfloor \frac{x}{T_i} \rfloor$ đơn vị cơ giáp. Vì các máy hoạt động độc lập và song song, tổng sản lượng của toàn bộ xưởng trong x giây sẽ là tổng sản lượng của từng máy cộng lại. *Lưu ý quan trọng:* Trong quá trình cộng dồn, nếu tổng sản lượng đã đạt hoặc vượt mức M yêu cầu, ta nên kết thúc sớm (`return true;`). Điều này vừa giúp tối ưu thời gian chạy, vừa tránh được lỗi tràn kiểu số nguyên (Overflow) nếu x quá lớn và các T_i quá nhỏ.

```
int n, m;
int t[100005];

bool check(int x) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += (x / t[i]);
        if (sum >= m) {
            return true;
        }
    }
    return false;
}
```

Bước 2: Tìm kiếm nhị phân thời gian x Thời gian tối thiểu để hoàn thành là 1 giây. Thời gian tối đa (Trường hợp xấu nhất) là khi ta chỉ có duy nhất 1 máy tính hoạt động và máy đó mất 10^9 giây để làm 1 sản phẩm, đồng thời ta cần 10^9 sản phẩm. Khi đó thời gian sẽ là $10^9 \times 10^9 = 10^{18}$ giây. Do đó, ta thiết lập cận trên `right = 1e18`.

```
void solve() {
    int left = 1, right = 1e18, ans = right;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}
```

3. Mã nguồn hoàn chỉnh Vì thời gian x và tổng sản lượng đều có thể lên tới mức 10^{18}, ta bắt buộc phải khai báo tất cả các biến liên quan bằng kiểu `long long`. Độ phức tạp thuật toán là $O(N \log(\text{Max\_Time}))$, tương đương khoảng 60×10^5 phép tính, chạy vô tư trong 1.0s.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int t[100005];

bool check(int x) {
    int sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += (x / t[i]);
        if (sum >= m) {
            return true;
        }
    }
    return false;
}

void solve() {
    int left = 1, right = 1e18, ans = right;

    while (left <= right) {
        int mid = (left + right) / 2;
        if (check(mid)) {
            ans = mid;
            right = mid - 1;
        } else {
            left = mid + 1;
        }
    }
    cout << ans << '\n';
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> m) {
        for (int i = 1; i <= n; i++) {
            cin >> t[i];
        }
        solve();
    }

    return 0;
}
```

Bài 20: Tổng M cặp lớn nhất

1. Phân tích bài toán

- **Cạm bẫy:** Có tới N^2 cặp, với $N = 10^5$ thì $N^2 = 10^{10}$. Việc tạo ra tất cả các cặp rồi sắp xếp lại để lấy M cặp lớn nhất là điều bất khả thi về cả thời gian (TLE) lẫn bộ nhớ (MLE).
- **Ý tưởng đột phá:** Thay vì đi tìm trực tiếp M cặp đó là gì, ta sẽ đặt câu hỏi ngược lại: *"Giá trị của cặp nhỏ nhất trong số M cặp lớn nhất này là bao nhiêu?"*. Gọi giá trị ngưỡng đó là X.
- **Tính đơn điệu:** Nếu ta chọn một mức X càng lớn, số lượng cặp $(A_i + A_j) \geq X$ sẽ càng ít. Ta có thể dùng Tìm kiếm nhị phân để tìm giá trị X lớn nhất sao cho số lượng cặp có tổng $\geq X$ đạt ít nhất là M.

2. Xây dựng từng bước

Bước 1: Sắp xếp & Hàm đếm số cặp Sau khi sắp xếp mảng A tăng dần, để đếm số lượng cặp có tổng $\geq X$, ta duyệt qua từng phần tử A_i. Ta cần tìm số lượng A_j thỏa mãn $A_j \geq X - A_i$. Do mảng đã sắp xếp, ta dùng `lower_bound` để tìm vị trí đầu tiên thỏa mãn. Mọi phần tử từ vị trí đó đến cuối mảng đều hợp lệ.

```
int n, m;
int a[100005];
int pre[100005];

int count_pairs(int x) {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(a + 1, a + n + 1, x - a[i]) - a;
        if (pos <= n) {
            cnt += (n - pos + 1);
        }
    }
    return cnt;
}
```

Bước 2: Tìm kiếm nhị phân ngưỡng X Tổng nhỏ nhất của một cặp là 2 (nếu $A_i = 1$). Tổng lớn nhất của một cặp là $2 \cdot 10^5$ (nếu $A_i = 10^5$). Ta tiến hành chặt nhị phân trong khoảng này để tìm ra ngưỡng X.

```
int find_threshold() {
    int left = 1, right = 200005, X = 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (count_pairs(mid) >= m) {
            X = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return X;
}
```

Bước 3: Tính tổng chính xác của M cặp Sau khi tìm được ngưỡng X, ta không thể đơn thuần lấy tất cả các cặp $\geq X$ vì có thể có rất nhiều cặp có tổng bằng đúng X, khiến số lượng vượt quá M. Cách xử lý an toàn nhất:

- Lấy toàn bộ các cặp có tổng **lớn hơn hẳn** X (tức là $\geq X + 1$). Ta đếm số lượng các cặp này (gọi là `cnt_greater`) và tính tổng của chúng thông qua Mảng cộng dồn (`Prefix Sum`).
- Số lượng cặp còn thiếu để đủ M cặp sẽ là $M -$ `cnt_greater`. Các cặp còn thiếu này chắc chắn đều có giá trị bằng đúng X.

```
void solve() {
    int X = find_threshold();

    int total_sum = 0;
    int cnt_greater = 0;

    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(a + 1, a + n + 1, X + 1 - a[i]) - a;
        if (pos <= n) {
            int count = n - pos + 1;
            cnt_greater += count;
            total_sum += count * a[i] + (pre[n] - pre[pos - 1]);
        }
    }

    total_sum += (m - cnt_greater) * X;
    cout << total_sum << '\n';
}
```

3. Mã nguồn hoàn chỉnh Giới hạn M có thể lên tới 10^{10} và tổng trọng số M cặp có thể tiệm cận $2 \cdot 10^{15}$. Do đó toàn bộ chương trình phải chạy trên nền kiểu `long long`. Độ phức tạp là $O(N \log N \log(\max A))$, chạy cực kỳ nhanh và mượt mà.

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, m;
int a[100005];
int pre[100005];

int count_pairs(int x) {
    int cnt = 0;
    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(a + 1, a + n + 1, x - a[i]) - a;
        if (pos <= n) {
            cnt += (n - pos + 1);
        }
    }
    return cnt;
}

int find_threshold() {
    int left = 1, right = 200005, X = 1;
    while (left <= right) {
        int mid = (left + right) / 2;
        if (count_pairs(mid) >= m) {
            X = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return X;
}

void solve() {
    int X = find_threshold();

    int total_sum = 0;
    int cnt_greater = 0;

    for (int i = 1; i <= n; i++) {
        int pos = lower_bound(a + 1, a + n + 1, X + 1 - a[i]) - a;
        if (pos <= n) {
            int count = n - pos + 1;
            cnt_greater += count;
            total_sum += count * a[i] + (pre[n] - pre[pos - 1]);
        }
    }

    total_sum += (m - cnt_greater) * X;
    cout << total_sum << ’\n’;
}

signed main() {
    ios_base::sync_with_stdio(false); cin.tie(0); cout.tie(0);

    if (cin >> n >> m) {
        for (int i = 1; i <= n; i++) {
            cin >> a[i];
        }

        sort(a + 1, a + n + 1);

        for (int i = 1; i <= n; i++) {
            pre[i] = pre[i - 1] + a[i];
        }

        solve();
    }

    return 0;
}
```